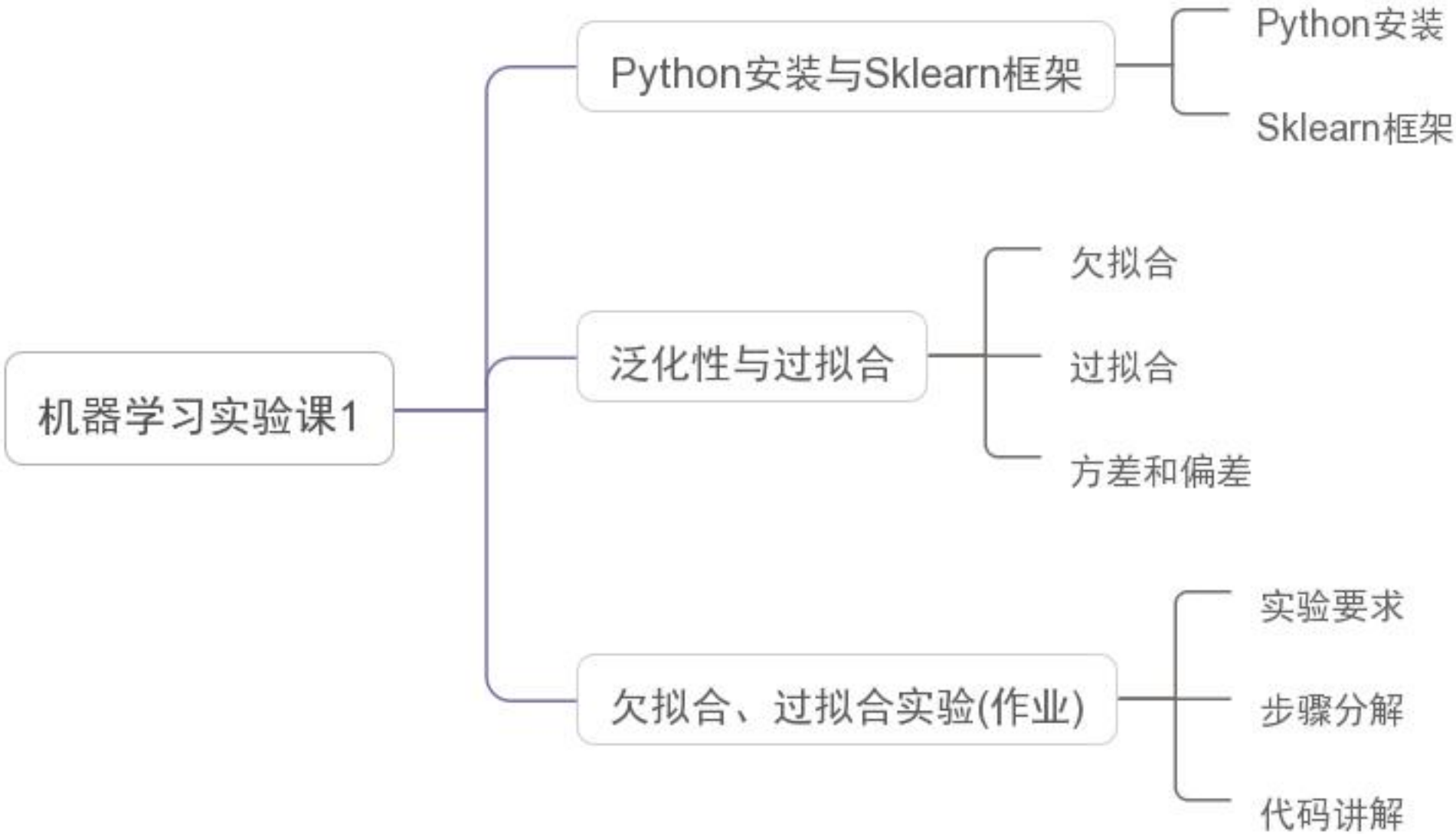
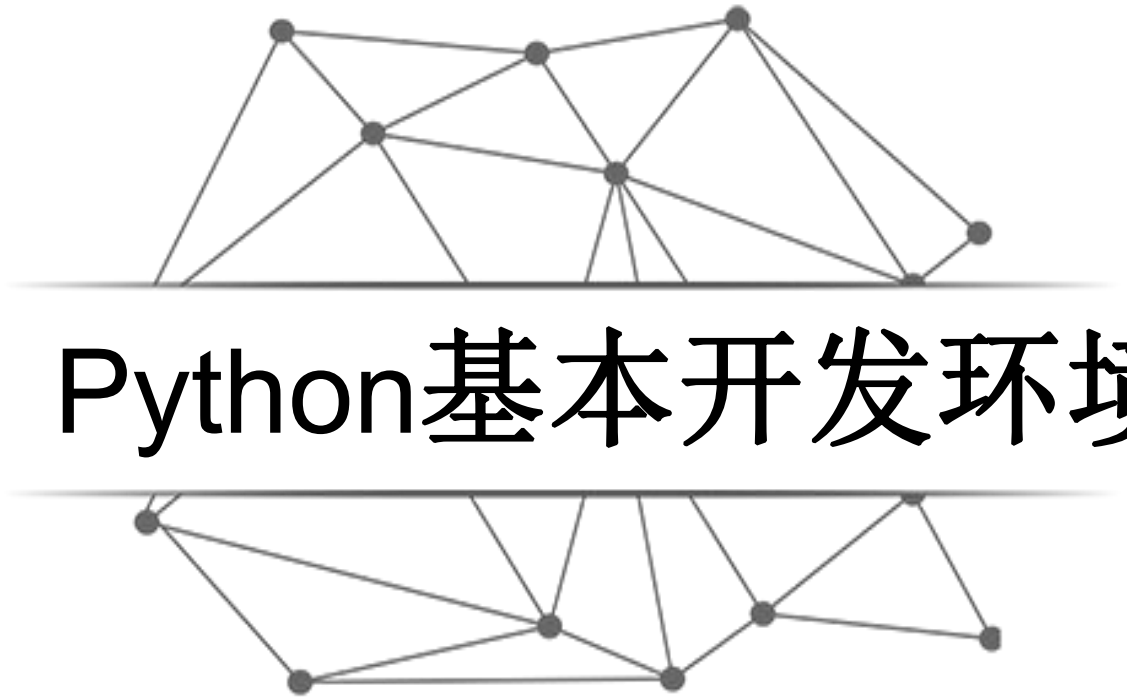
# 机器学习实验课1

*ML01*




致谢：梁威鹏


机器学习实验课1
Python安装与Sklearn框架
Python安装
Sklearn框架
泛化性与过拟合
欠拟合
过拟合
方差和偏差
欠拟合、过拟合实验(作业)
实验要求
步骤分解
代码讲解



# Python基本开发环境

# Python安装

- 纯Python：https://www.python.org/downloads/

- 集成环境Anaconda：
  https://www.anaconda.com/products/distribution

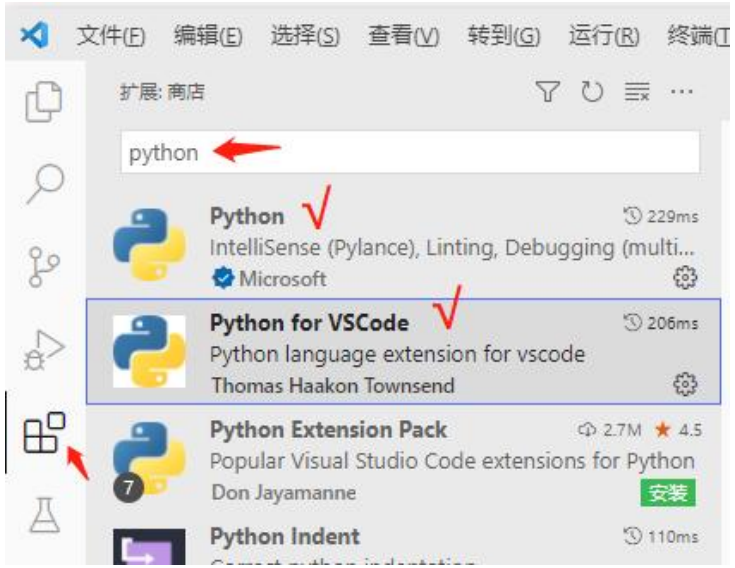
- 建议安装Python 3.7-3.9的版本

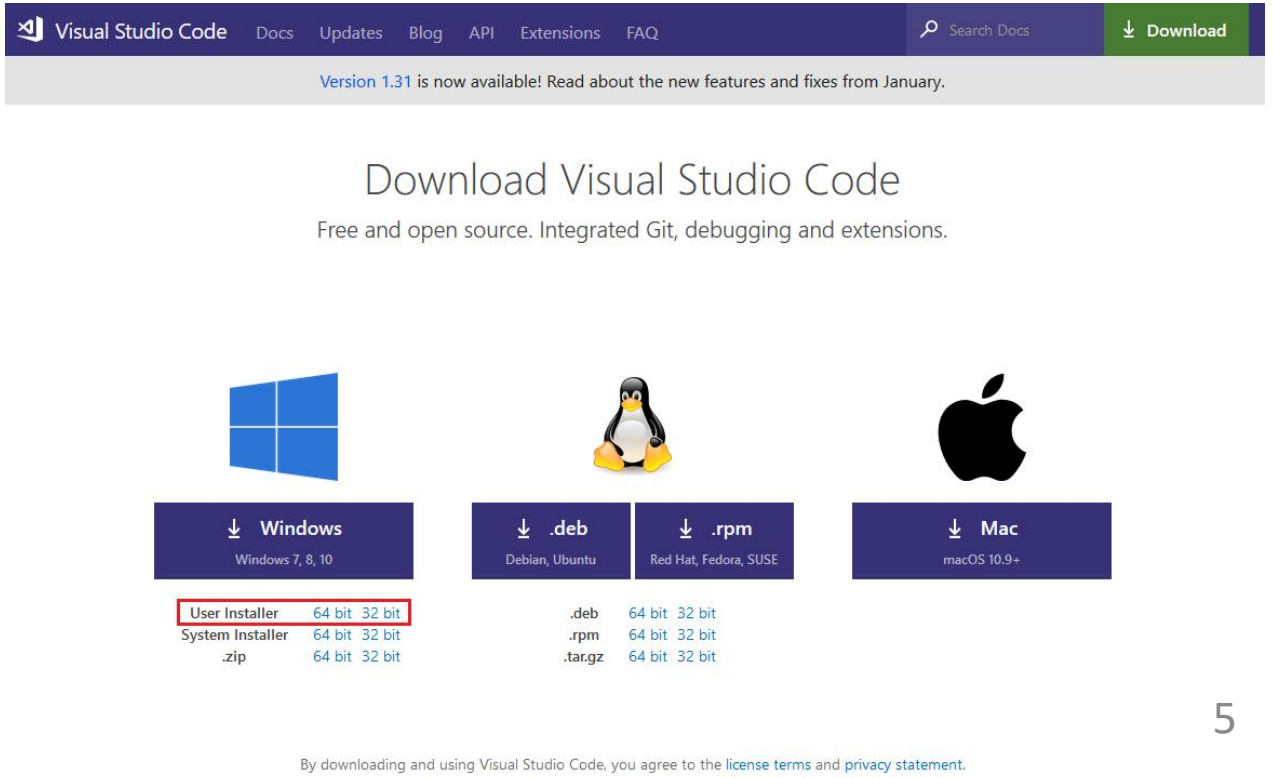
# Python高级开发环境VSCode

## 安装VSCode

- 下载地址：**https://code.visualstudio.com**

- 工具大小约 **50MB**

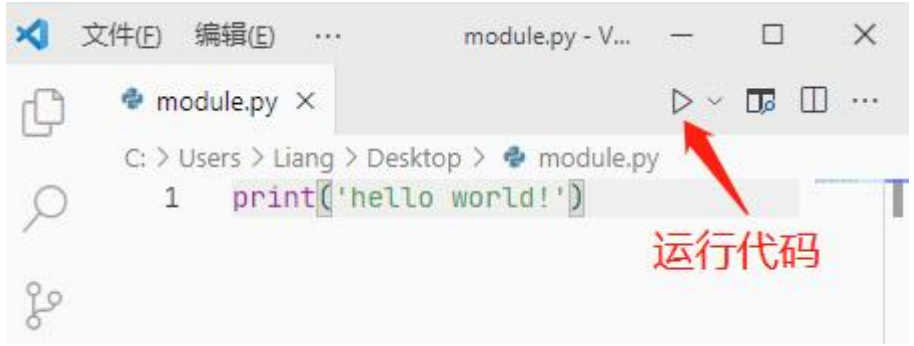


# Python的两种编程方式

## 交互式和文件式

- 交互式：对每个输入语句即时运行结果，适合语法练习
- 文件式：批量执行一组语句并运行结果，编程的主要方式。






# Python其它开发环境

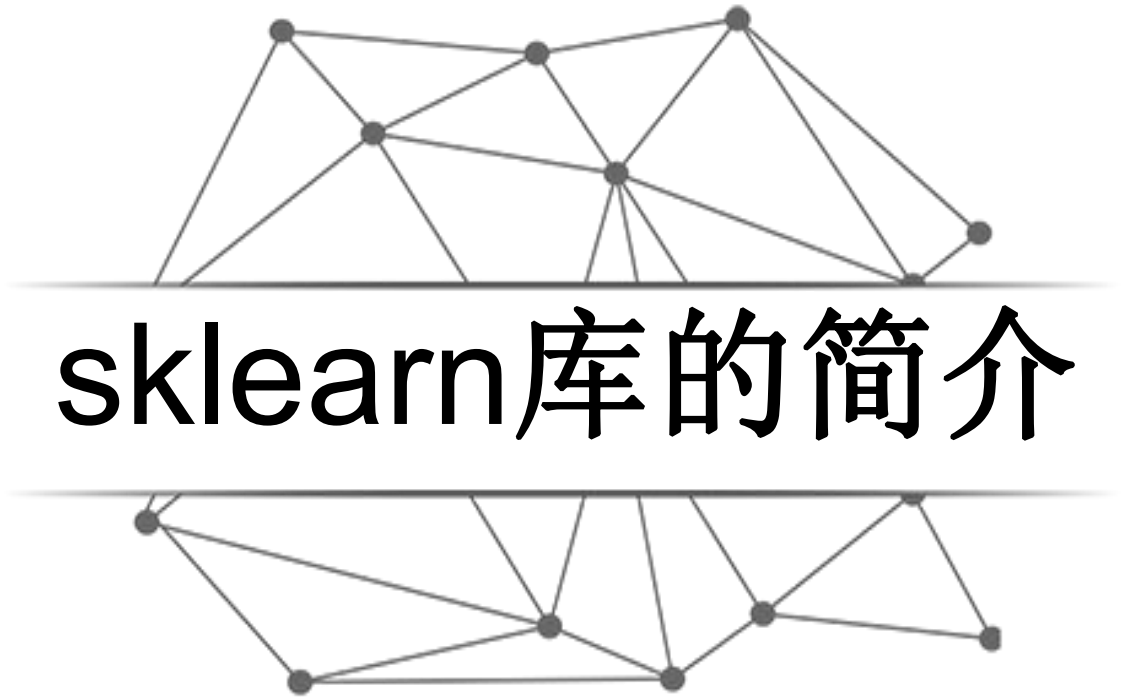
- **Pycharm**

- **Jupyter Notebook**

  - pip install juypter
  - cmd输入> jupyter notebook

- **Anaconda**

**IDE(集成开发环境)只是工具，真大佬用记事本也可以写程序；**



# sklearn库的简介

# Sklearn库

sklearn是scikit-learn的简称，是一个基于Python的第三方模块。sklearn库集成了一些常用的机器学习方法，在进行机器学习任务时，并不需要实现算法，只需要简单的调用sklearn库中提供的模块就能完成大多数的**机器学习**任务。

sklearn库是在Numpy、Scipy和matplotlib的基础上开发而成的，因此在介绍sklearn的安装前，需要先安装这些依赖库。

# 相关依赖库

Numpy（Numerical Python的缩写）是一个开源的Python科学计算库。在Python中虽然提供了list容器和array模块，但这些结构并不适合于进行**数值计算**，因此需要借助于Numpy库创建常用的数据结构（如：多维数组，矩阵等）以及进行常用的科学计算（如：矩阵运算）。

Scipy库是sklearn库的基础，它是基于Numpy的一个集成了多种数学算法和函数的Python模块。它的不同子模块有不同的应用，如：**积分、插值、优化和信号处理**等。

matplotlib是基于Numpy的一套Python工具包，它提供了大量的**数据绘图**工具，主要用于绘制一些统计图形，将大量的数据转换成更加容易被接受的图表。

（注意要先安装numpy再安装matplotlib库）

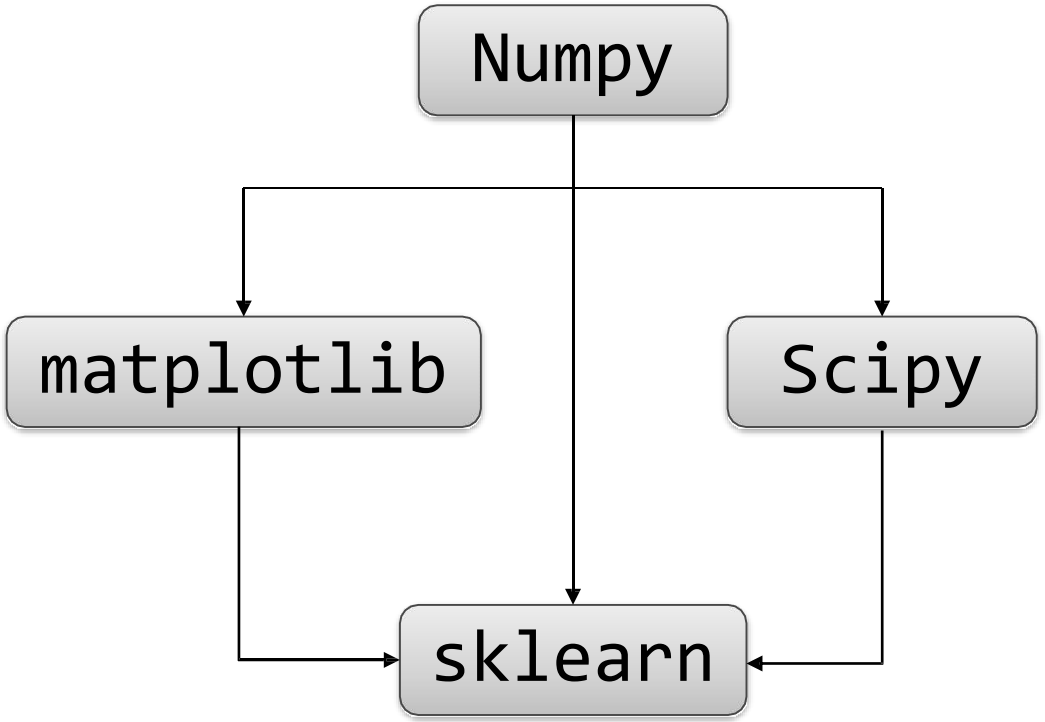


# sklearn库的安装

# 安装顺序

**安装顺序如下：**

1. Numpy库
2. Scipy库
3. matplotlib库
4. sklearn库



# 安装步骤：




**安装步骤如下：**

1. 配置pip国内镜像；(重要)

   (可参考链接：参考链接)

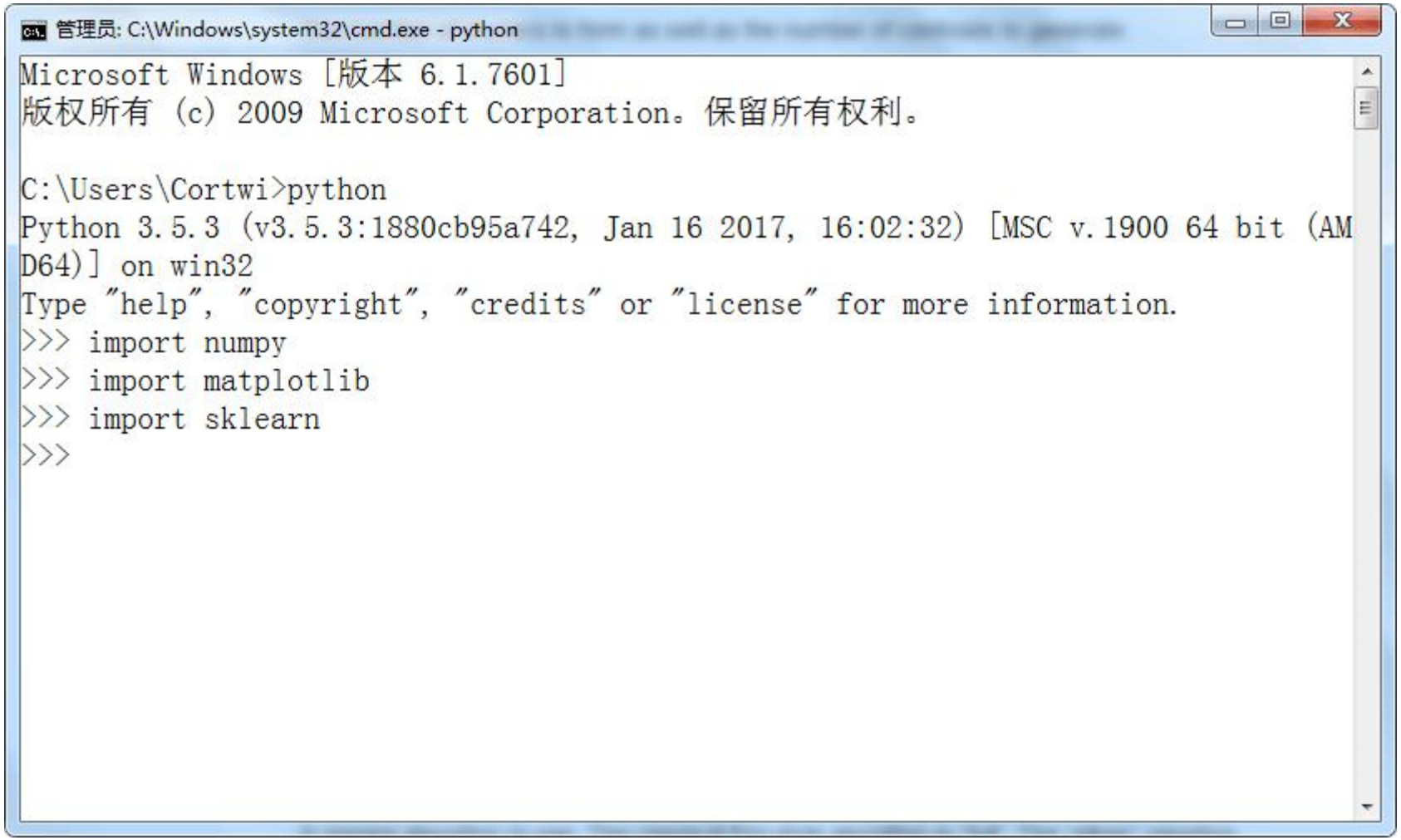
1. 打开cmd命令行或者VsCode终端，输入以下命令；

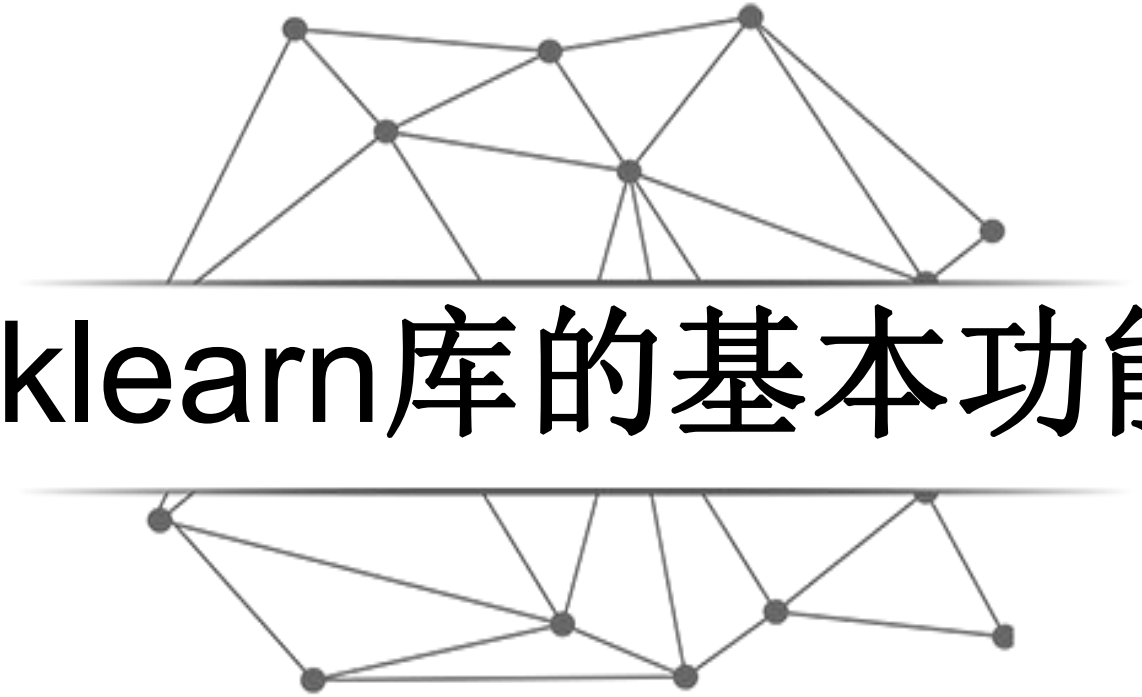
2. 安装Numpy——> `pip install numpy`

3. 安装matplotlib库——> `pip install matplotlib`

4. 安装sklearn库——> `pip install scikit-learn`

# 安装测试


管理员: C:\Windows\system32\cmd.exe - python
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。
C:\Users\Cortwi>python
Python 3.5.3 (v3.5.3:1880cb95a742, Jan 16 2017, 16:02:32) [MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>> import matplotlib
>>> import sklearn
>>>



# sklearn库的基本功能

# sklearn库的基本功能

sklearn库的共分为6大部分，分别用于完成**分类任务、回归任务、 聚类任务、降维任务、模型选择**以及**数据的预处理**。

| | 应用<br>（**Applications**） | 算法<br>（**Algorithm**） |
|---|---|---|
| 分类<br>（Classification） | 异常检测，图像识别，等 | KNN, SVM，etc. |
| 聚类<br>（Clustering） | 图像分割，群体划分，等 | K-Means，谱聚类, etc. |
| 回归<br>（Regression） | 价格预测，趋势预测，等 | 线性回归，SVR，etc. |
| 降维<br>（Dimension Reduction） | 可视化， | PCA，NMF，etc. |

# 分类任务

| 分类模型 | 加载模块 |
| --- | --- |
| 最近邻算法 | neighbors.NearestNeighbors |
| 支持向量机 | svm.SVC |
| 朴素贝叶斯 | naive_bayes.GaussianNB |
| 决策树 | tree.DecisionTreeClassifier |
| 集成方法 | ensemble.BaggingClassifier |
| 神经网络 | neural_network.MLPClassifier |

# 回归任务

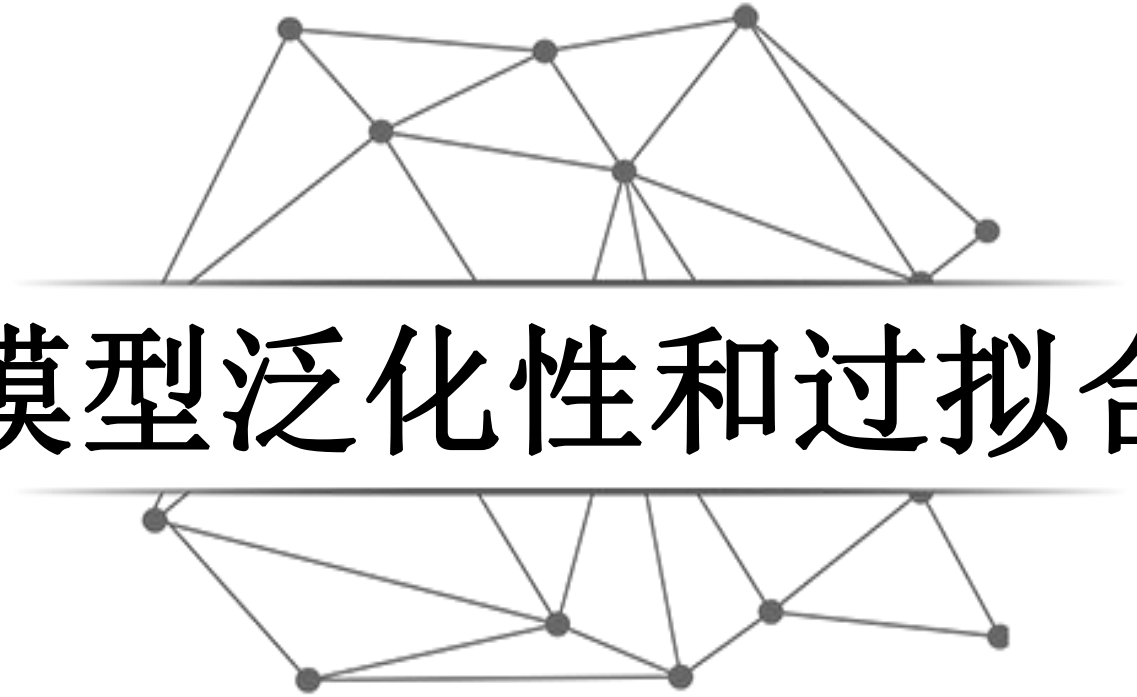
| 回归模型 | 加载模块 |
| --- | --- |
| 岭回归 | linear_model.Ridge |
| Lasso回归 | linear_model.Lasso |
| 弹性网络 | linear_model.ElasticNet |
| 最小角回归 | linear_model.Lars |
| 贝叶斯回归 | linear_model.BayesianRidge |
| 逻辑回归 | linear_model.LogisticRegression |
| 多项式回归 | preprocessing. PolynomialFeatures |

# 聚类任务

| 聚类方法 | 加载模块 |
| --- | --- |
| K-means | cluster.KMeans |
| AP聚类 | cluster.AffinityPropagation |
| 均值漂移 | cluster.MeanShift |
| 层次聚类 | cluster.AgglomerativeClustering |
| DBSCAN | cluster.DBSCAN |
| BIRCH | cluster.Birch |
| 谱聚类 | cluster.SpectralClustering |

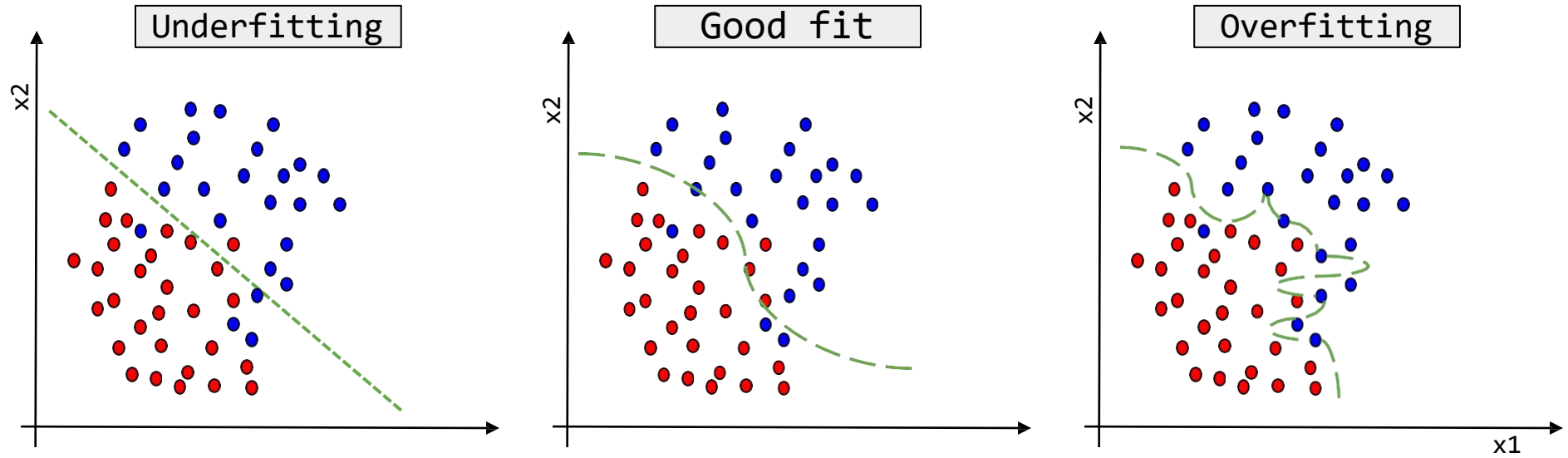
# 降维任务

| 降维方法 | 加载模块 |
| --- | --- |
| 主成分分析 | decomposition.PCA |
| 截断SVD和LSA | decomposition.TruncatedSVD |
| 字典学习 | decomposition.SparseCoder |
| 因子分析 | decomposition.FactorAnalysis |
| 独立成分分析 | decomposition.FastICA |
| 非负矩阵分解 | decomposition.NMF |
| LDA | decomposition.LatentDirichletAllocation |

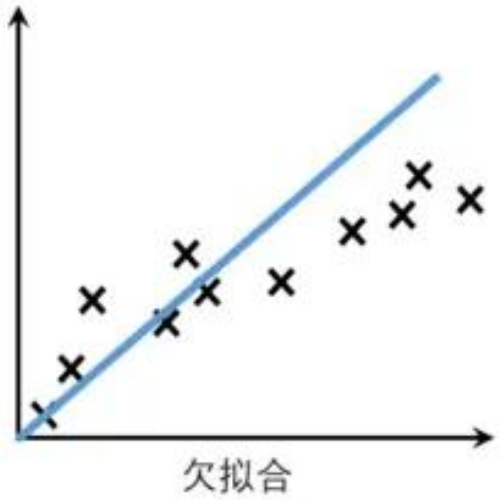


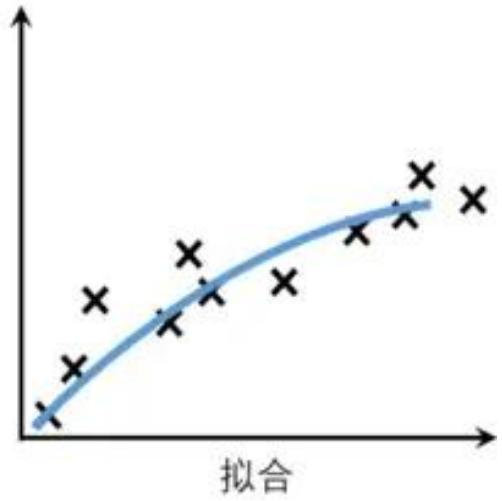
# 模型泛化性和过拟合

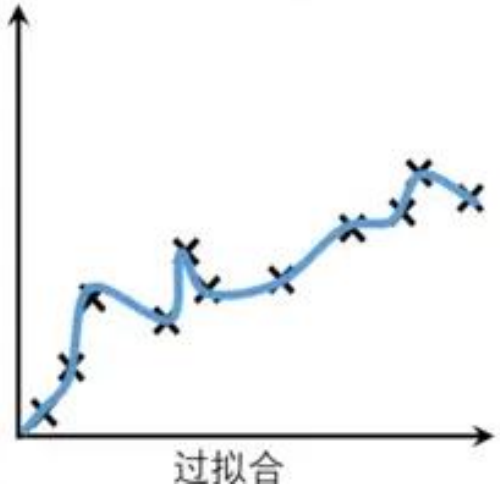
# 分类问题中的拟合

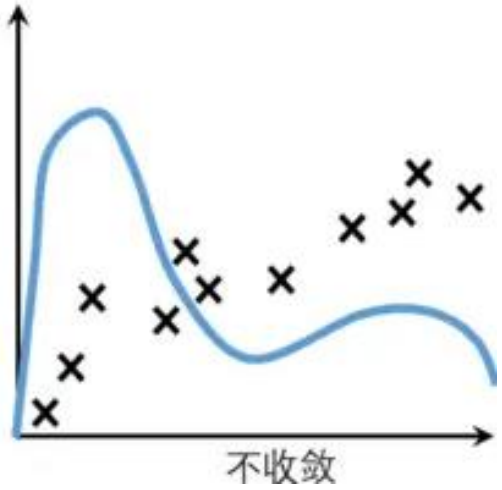


# 回归问题中的拟合



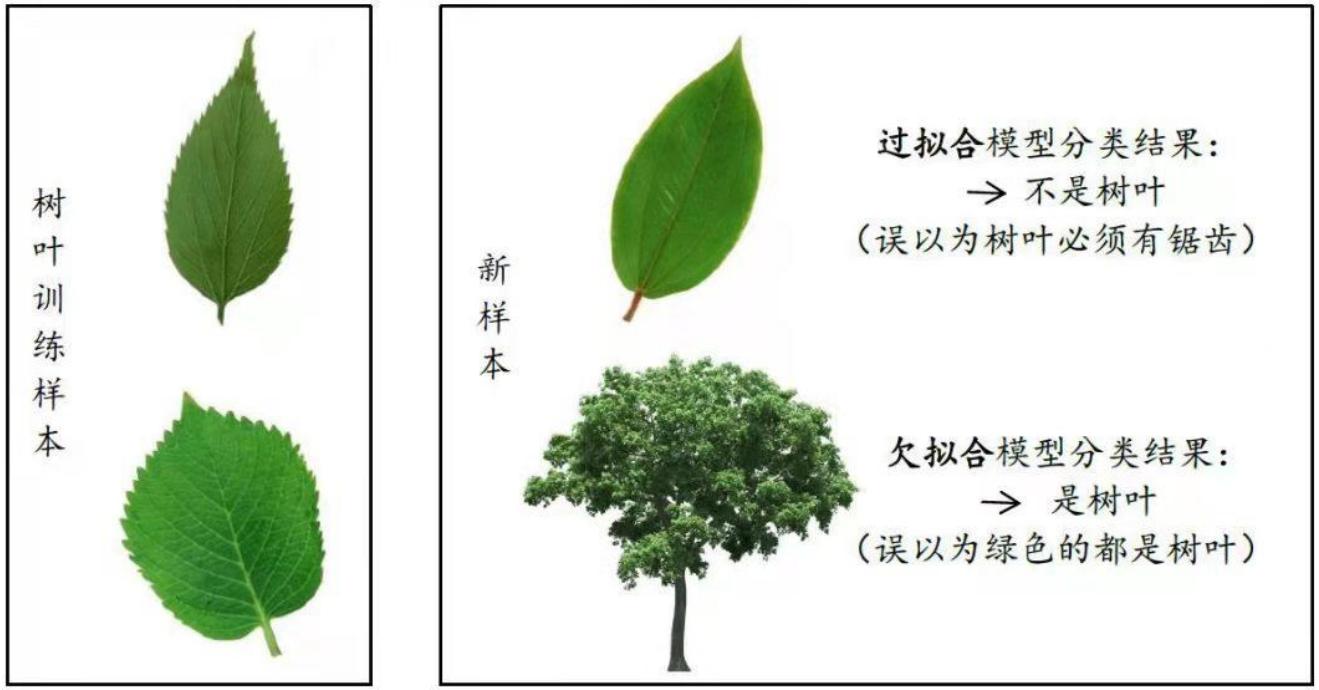



- 回归问题中的过拟合



- 分类问题中的过拟合




过拟合、欠拟合的直观类比

# 偏差和方差

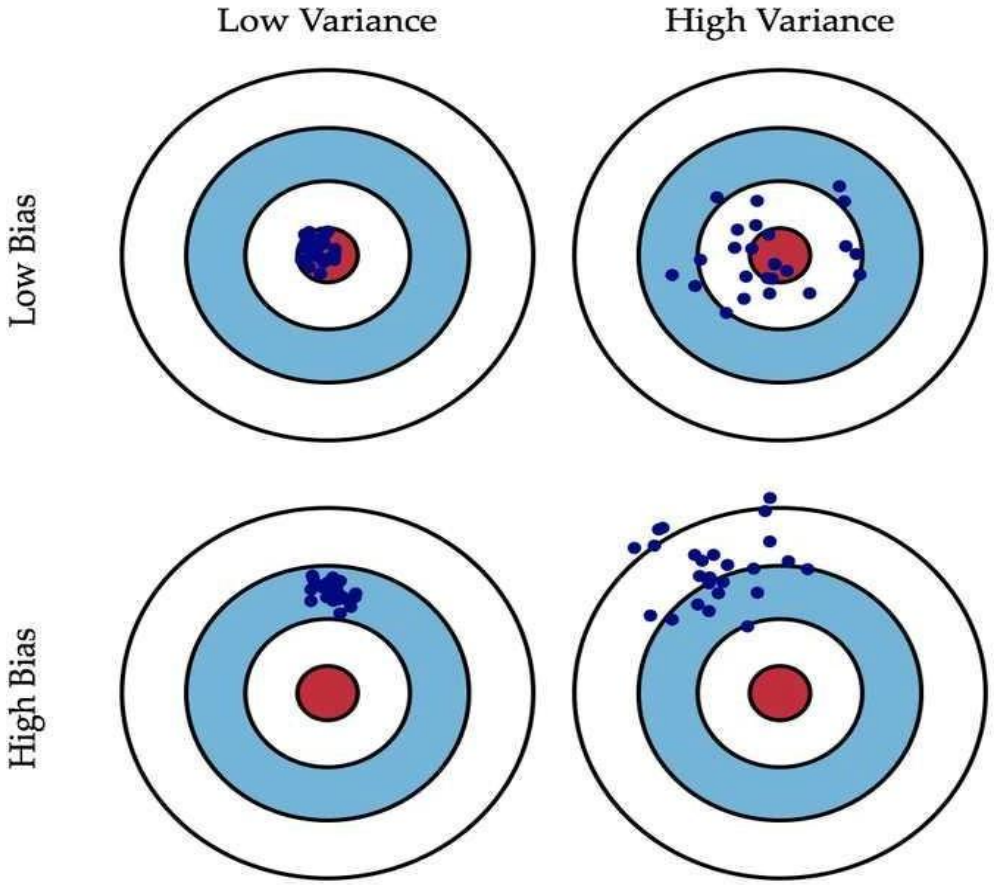


**方差Variance：**

描述的是预测值的变化范围，离散程度， 也就是离其期望值的距离。方差越大，数据的分布越分散，如右图右列所示。

**偏差Bias：**

描述的是预测值（估计值）的期望与真实值之间的差距。偏差越大，越偏离真实数据，如右图第二行所示。




- **欠拟合：** 模型不能适配训练样本，有一个很大的偏差。
- **过拟合：** 模型很好的适配训练样本，但在测试集上表现很糟，有一个很大的方差。

# 欠拟合的处理

**1.添加新特征**

当特征不足或者现有特征与样本标签的相关性不强时，模型容易出现欠拟合。通过挖掘组合特征等新的特征，往往能够取得更好的效果。

**2.增加模型复杂度**

简单模型的学习能力较差，通过增加模型的复杂度可以使模型拥有更强的拟合能 力。例如，在线性模型中添加高次项，在神经网络模型中增加网络层数或神经元 个数等。

**3.减小正则化系数**

正则化是用来防止过拟合的，但当模型出现欠拟合现象时，则需要有针对性地减小正则化系数。

# 过拟合的处理

**1.获得更多的训练数据**

使用更多的训练数据是解决过拟合问题最有效的手段，因为更多的样本能够让模型学习到更多更有效的特征，减小噪声的影响。

**2.降维**

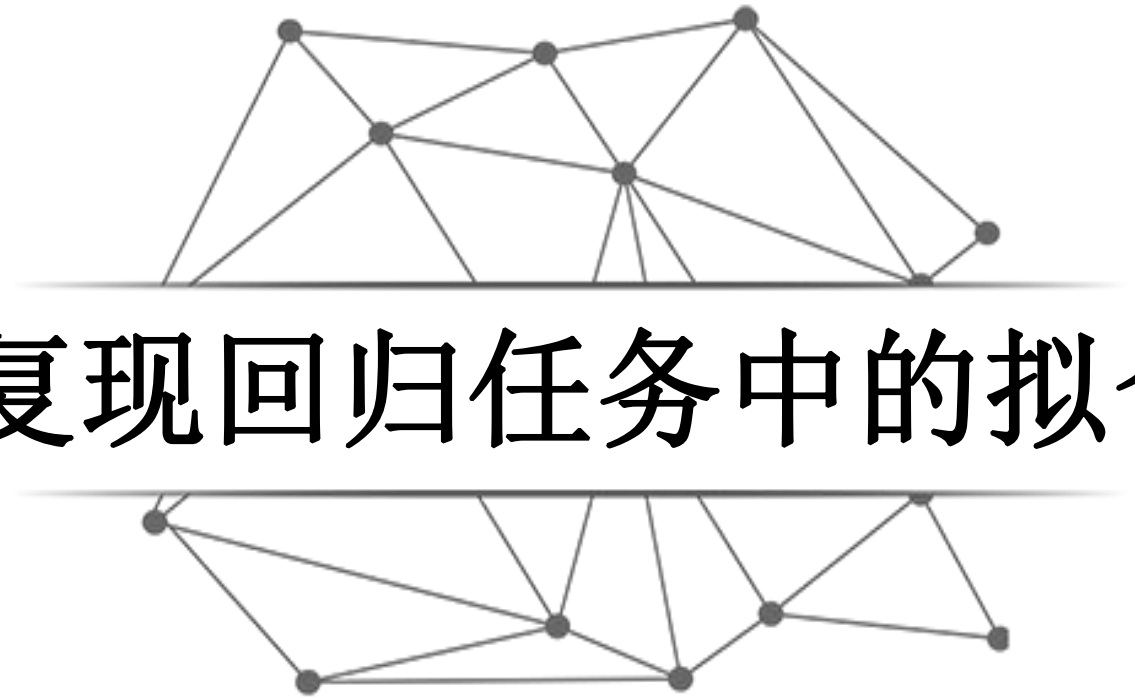
即丢弃一些不能帮助我们正确预测的特征。可以是手工选择保留哪些特征，或者使用一些模型选择的算法来帮忙（例如PCA）。

**3.正则化**

正则化(regularization)的技术，保留所有的特征，但是减少参数的大小（magnitude），它可以改善或者减少过拟合问题。

**4.集成学习方法**

集成学习是把多个模型集成在一起，来降低单一模型的过拟合风险。



# 复现回归任务中的拟合

# 实验步骤：

**实验步骤如下：**

1. 生成随特定函数分布的几百个随机数据点，例如y=sin(x)；

2. 将其分为训练集和测试集，并可视化数据点；

3. 利用sklearn框架拟合；
   - 提示：sklearn.preprocessing.PolynomialFeatures()
   - 官方文档: https://scikit-learn.org/stable/
   - 中文文档：https://sklearn.apachecn.org/

4. 改变传入PolynomialFeatures()中的系数来调整曲线拟合程度;
5. 可视化拟合结果；（三幅图：欠拟合、过拟合、正常拟合）

## 1. 生成随特定函数分布的几百个随机数据点

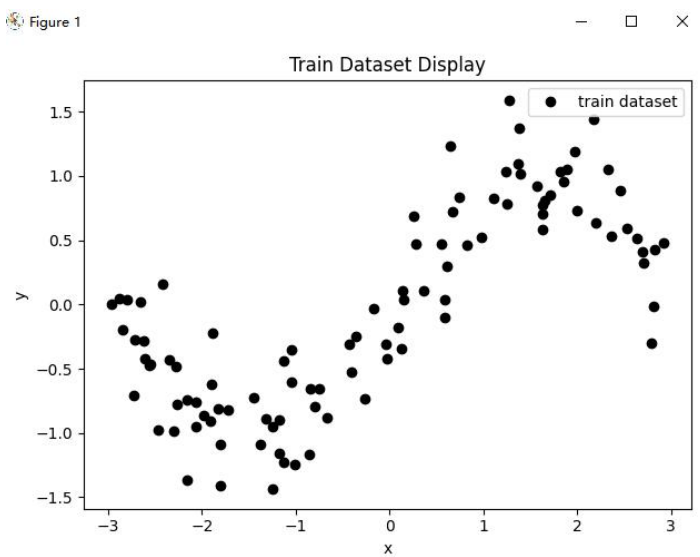
```python
# -*- coding: utf-8
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures

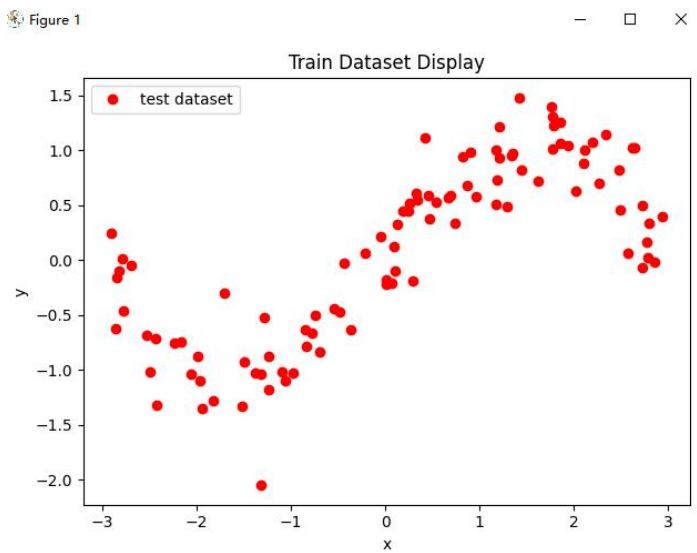
# 设置随机种子数
rnd = np.random.RandomState(42)

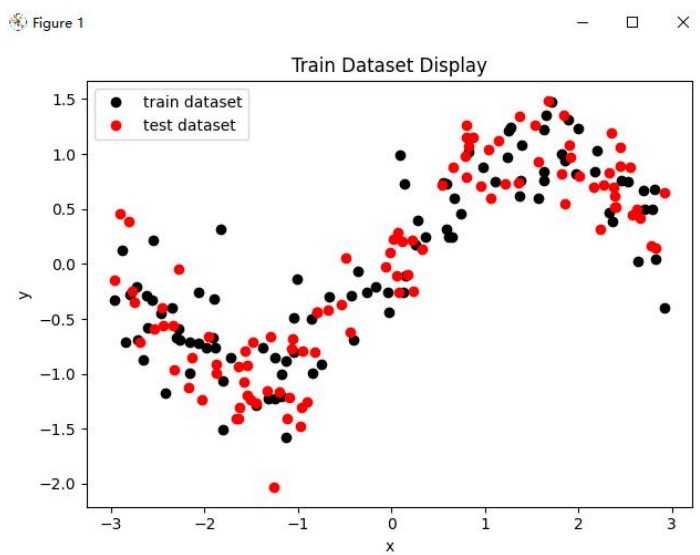
# x是100个(-3,3)之间随机数
# 训练集
x_train = rnd.uniform(-3,3,size=100)
y_train = np.sin(x_train) + rnd.normal(size=len(x_train)) / 3
# 测试集
x_test = rnd.uniform(-3,3,size=100)
y_test = np.sin(x_test) + rnd.normal(size=len(x_test)) / 3
```

## 2. 分为训练集和测试集，并可视化数据点

```python
# 绘制训练集和测试集
fig,ax1 = plt.subplots(1)
ax1.plot(x_train,y_train,'o',c='k',label='train dataset')
ax1.plot(x_test,y_test,'o',c='r',label='test dataset')
ax1. legend(loc="best")
#其他图形选项
ax1.set_ylabel("y")
ax1.set_xlabel("x")
ax1.set_title("Train Dataset Display")
plt.tight_layout()
plt.show()
```



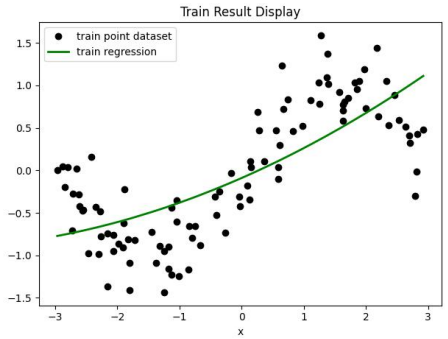
## 3.sklearn框架拟合数据

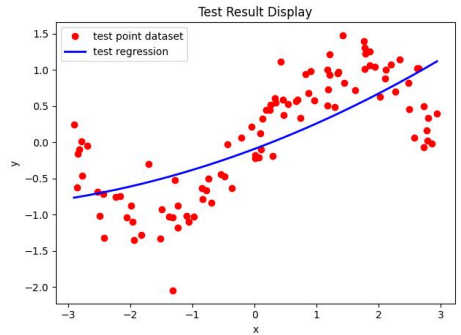
```
33  #后续fit的输入x需要是二维数据，因此需要改变一下x的维度
34  x_train_reshape = x_train.reshape(-1,1)
35  x_test_reshape = x_test.reshape(-1,1)
36
37  # 实例化多项式转换模型
38  poly = PolynomialFeatures(degree=4)
39  x_train_poly = poly.fit_transform(x_train_reshape)
40
41  #实例化线性模型
42  lr = LinearRegression()
43  #在训练集上训练
44  lr.fit(x_train_poly, y_train)
45  y_predict = lr.predict(x_train_poly)
46
47  #在测试集上训练
48  x_test_poly = poly.fit_transform(x_test_reshape)
49  y_predict_test = lr.predict(x_test_poly)
```

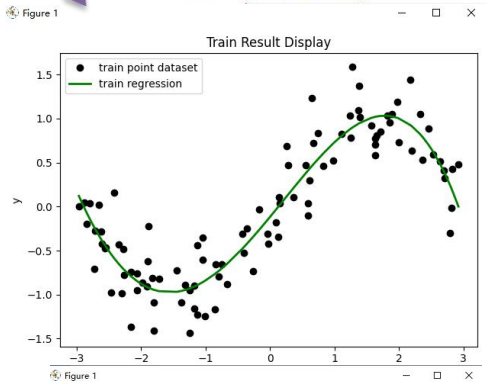
## 4. 可视化拟合结果

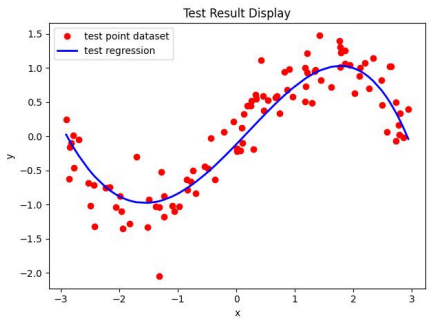
```
55  # 可视化图形
56  fig,ax1 = plt.subplots(1)
57  ax1.plot(x_train,y_train,'o',c='k',label='train point dataset')
58  ax1.plot(np.sort(x_train),y_predict[np.argsort(x_train)],linewidth=2,c='g',label='train regression')
59  ax1. legend(loc="best")
60  #其他图形选项
61  ax1.set_ylabel("y")
62  ax1.set_xlabel("x")
63  ax1.set_title("Train Result Display")
64  plt.tight_layout()
65  plt.show()
66  plt.close()
67
68
69  fig,ax1 = plt.subplots(1)
70  ax1.plot(x_test,y_test,'o',c='r',label='test point dataset')
71  ax1.plot(np.sort(x_test),y_predict_test[np.argsort(x_test)],linewidth=2,c='b',label='test regression')
72  ax1. legend(loc="best")
73  #其他图形选项
74  ax1.set_ylabel("y")
75  ax1.set_xlabel("x")
76  ax1.set_title("Test Result Display")
77  plt.tight_layout()
78  plt.show()
79  plt.close()
```

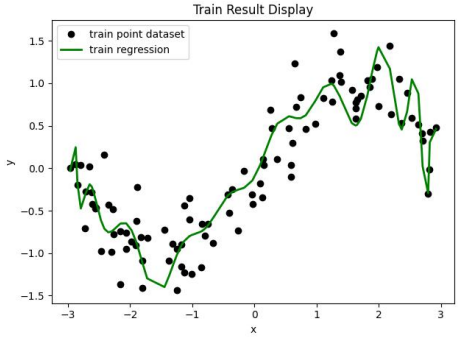
degree = 2

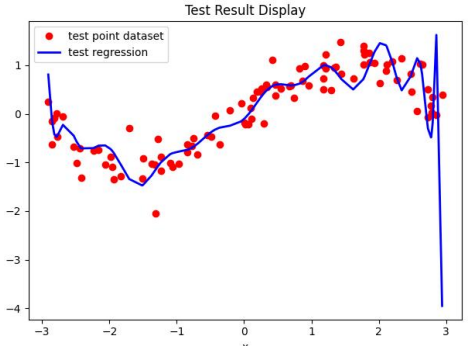



degree = 25

## polynomial features 多项式特征：

- 该预处理器将输入数据矩阵转换为给定度的新数据矩阵

```
In [1]: from sklearn.preprocessing import PolynomialFeatures
        import numpy as np
        X = np.arange(6).reshape(3, 2)

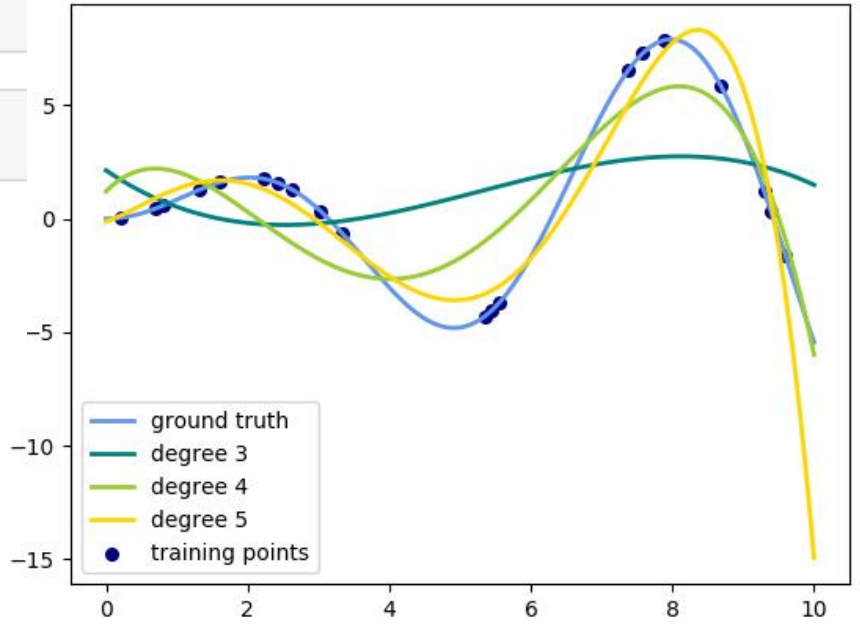
In [2]: X

Out[2]: array([[0, 1],
               [2, 3],
               [4, 5]])

In [3]: poly = PolynomialFeatures(degree=2)
        poly.fit_transform(X)

Out[3]: array([[ 1.,  0.,  1.,  0.,  0.,  1.],
               [ 1.,  2.,  3.,  4.,  6.,  9.],
               [ 1.,  4.,  5., 16., 20., 25.]])
```

X的特征 $[x_1, x_2]$




新的X的特征 $[1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$

# THANK YOU FOR WATCHING

梁威鹏